

15.3: Operations on Structures

[This section corresponds roughly to K&R Sec. 6.2]

There is a relatively small number of operations which C directly supports on structures. As we've seen, we can define structures, declare variables of structure type, and select the members of structures. We can also assign entire structures: the expression

```
c1 = c2
```

would assign *all* of `c2` to `c1` (both the real and imaginary parts, assuming the preceding declarations). We can also pass structures as arguments to functions, and declare and define functions which return structures. But to do anything else, we typically have to write our own code (often as functions). For example, we could write a function to add two complex numbers:

```
struct complex
cpx_add(struct complex c1, struct complex c2)
{
    struct complex sum;
    sum.real = c1.real + c2.real;
    sum.imag = c1.imag + c2.imag;
    return sum;
}
```

We could then say things like

```
c1 = cpx_add(c2, c3)
```

(Two footnotes: Typically, you do need a temporary variable like `sum` in a function like `cpx_add` above that returns a structure. In C++, it's possible to couple your own functions to the built-in operators, so that you could write `c1 = c2 + c2` and have it call your complex add function.)

One more thing you can do with a structure is initialize a structure variable while declaring it. As for array initializations, the initializer consists of a comma-separated list of values enclosed in braces `{}`:

```
struct complex c1 = {1, 2};
struct complex c2 = {3, 4};
```

Of course, the type of each initializer in the list must be compatible with the type of the corresponding structure member.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1996-1999 // [mail](#) [feedback](#)